Project Proposal: Development of a Minimal x86-64 Operating System Kernel

1. Project Objective

1.1 Project Goal

The goal of this project is to develop a minimal 64-bit operating system kernel in C and Assembly that boots in QEMU using Multiboot2 and provides an interactive terminal, memory management, interrupt handling, keyboard input, and graphical framebuffer support. If time permits, an existing DOOM port will be integrated to demonstrate the kernel's capabilities.

The project is designed for a workload of 9 credit points (CP), equivalent to approximately 270 hours of work, including class time.

1.2 SMART Evaluation

Criterion	Evaluation
Specific	Develop a minimal x86-64 kernel with a boot process, terminal, shell, memory management, interrupts, timing, keyboard input, and graphics.
Measurable	Verify functionality through QEMU demonstrations, automated or repeatable tests, and documented acceptance criteria.
Attractive / Accepted	The project combines operating systems, computer architecture, low-level programming, and graphics. Team members will agree on the core objectives and responsibilities.
Realistic	The existing bootloader, basic kernel output, and VGA text driver provide a starting point. Core functionality will take priority over optional features.
Scheduled	Complete the mandatory objectives by the end of the semester, with intermediate milestones and reserved integration time.

1.3 Acceptance Criteria

At the end of the semester, the following criteria will be evaluated with a yes-or-no result.

1. The kernel builds and boots in QEMU, displays diagnostic output, and provides an interactive shell that executes built-in commands.
2. Physical memory allocation, deallocation, and kernel heap operations pass repeatable tests.
3. CPU exception handlers provide useful diagnostic information, and the interrupt controller and timer operate correctly.
4. Keyboard input works, and the kernel can render graphics using a bootloader-provided framebuffer.
5. The kernel can load and read a WAD file from memory. If the stretch goal is achieved, a DOOM port runs with functional graphics and keyboard input.

The first four criteria define the core project deliverable. The fifth criterion is a stretch goal and does not determine whether the core project is complete.

2. Scope Definition

2.1 Included in Scope

The project includes the following components:

- Multiboot2 boot process and x86-64 kernel initialization.
- Cross-compilation toolchain, linker script, Makefile, and QEMU configuration.
- Serial output, logging, and GDB debugging.
- VGA text output and formatted kernel printing.
- Interactive shell with command parsing and built-in commands.
- PS/2 keyboard input.
- Framebuffer graphics and basic pixel operations.
- Global Descriptor Table (GDT), Task State Segment (TSS), and Interrupt Descriptor Table (IDT).
- CPU exception handlers and kernel panic diagnostics.
- Programmable Interrupt Controller (PIC) configuration and system timer.
- Physical memory management, virtual memory management, and kernel heap allocation.
- A simple read-only or in-memory filesystem.
- Loading and accessing a WAD file through a boot module.
- QEMU-based testing and final integration.
- Integration of an existing DOOM port, subject to available time.

2.2 Non-Goals

The following features are outside the mandatory project scope:

- A complete general-purpose operating system.
- Multitasking, process scheduling, and user-space process isolation.
- Networking and Ethernet drivers.
- USB support and a general-purpose USB input stack.
- Audio support.
- A complete disk driver or general-purpose disk filesystem.
- A graphical desktop environment or window manager.

- Broad compatibility with physical hardware.
- Writing a new game engine from scratch.

These features may be considered only if the core objectives have been completed.

2.3 Assumptions

- Development and testing will primarily take place in QEMU.
- The existing Multiboot2 bootloader will be retained.
- GRUB will load the kernel and provide the Multiboot2 information structure.
- The framebuffer and WAD file will be supplied through bootloader-supported mechanisms.
- The team has access to an x86-64 cross-compilation toolchain, assembler, linker, QEMU, and GDB.
- An existing DOOM port will be used if game integration is attempted.
- The team will prioritize a reliable and testable kernel over optional functionality.

2.4 Framework Conditions

Category	Specification
Workload	9 CP, approximately 270 hours
Architecture	x86-64
Programming languages	C and Assembly
Boot protocol	Multiboot2
Bootloader	GRUB
Emulator	QEMU
Debugging	GDB and serial output
Build system	Make and a cross-compilation toolchain
Version control	Shared Git repository
Collaboration	Regular meetings, agreed communication channels, code reviews, and documented interfaces
Documentation	Working paper, build instructions, milestone results, and test evidence

3. Rough Milestones

The milestones below are defined relative to the start of the project. Calendar dates should be aligned with the semester schedule. The final integration period is reserved for debugging, testing, and documentation.

Milestone 1 - Stable Boot and Development Environment

Target: End of Week 2

Description:

Establish a reproducible development environment and ensure that all team members can build and boot the same kernel.

Deliverables:

- Working x86-64 cross-compilation toolchain.
- Linker script and Makefile.
- Multiboot2 boot configuration.
- QEMU boot configuration.
- Serial output and basic logging.
- GDB debugging setup.
- Shared Git repository and agreed module interfaces.

Completion criterion: The documented build command successfully compiles the kernel, and `make run` boots it in QEMU and displays a message without immediately crashing.

Responsible roles: Build system and boot integration; kernel initialization; repository coordination.

Milestone 2 - Interactive Console and Shell

Target: End of Week 4

Description:

Provide a usable command-line interface for interacting with the kernel.

Deliverables:

- VGA text output with scrolling, backspace, and cursor control.
- `kprintf()` supporting `%s`, `%d`, `%x`, `%c`, and `%p`.
- Serial input and output.
- Line editor supporting typing, backspace, and Enter.
- Command parser and dispatcher.
- Built-in commands such as `help`, `echo`, and `clear`.

Completion criterion: The kernel accepts multiple commands, displays their results, and handles unknown commands without crashing.

Responsible roles: Terminal and input development; shell implementation; integration testing.

Milestone 3 - Memory Management and CPU Diagnostics

Target: End of Week 7

Description:

Implement essential memory-management and CPU exception-handling infrastructure.

Deliverables:

- Multiboot2 memory-map parser.
- Physical frame allocator for 4 KiB pages.
- Virtual memory mapping and unmapping.
- Kernel heap with `kmalloc()` and `kfree()`.
- GDT and TSS configuration.
- IDT and CPU exception handlers.
- PIC configuration and timer ticks.
- Kernel panic diagnostics and register dumps.

Completion criterion: Repeatable allocation and deallocation tests pass, timer ticks are generated, and a controlled CPU exception produces useful diagnostic output.

Responsible roles: Memory management; CPU architecture and interrupt handling; test integration.

Milestone 4 - Keyboard, Graphics, and File Loading

Target: End of Week 9

Description:

Provide the input, graphics, and file-access functionality needed for future game integration.

Deliverables:

- PS/2 keyboard driver with scancode translation and key-state handling.
- Framebuffer initialization using Multiboot2 information.
- Pixel drawing, screen clearing, and a basic graphics test.
- Back buffer for efficient rendering.
- In-memory file loading and reading.
- Read-only filesystem or memory-blob interface.
- WAD module loading configuration.

Completion criterion: A QEMU demonstration accepts keyboard input, renders a graphics test pattern, and reads a known file loaded into memory.

Responsible roles: Device drivers; graphics and input integration; filesystem and boot-module support.

Milestone 5 - Final Integration and Demonstration

Target: Final two weeks of the semester

Description:

Integrate the completed kernel components, execute the acceptance tests, and prepare the final demonstration.

Deliverables:

- Reproducible build and run instructions.
- Integrated kernel subsystems.
- Test results for all mandatory acceptance criteria.
- Documentation of known limitations and remaining issues.
- Final demonstration.
- DOOM integration, if feasible within the remaining time.

Completion criterion: The kernel satisfies the mandatory acceptance criteria, and the team can demonstrate the results from a clean checkout using the documented instructions.

Responsible roles: All team members, coordinated by an integration lead.

3.1 Individual Learning Objectives

Each team member will define an individual learning objective and contribute a verifiable result. Possible objectives include:

- Implementing and debugging x86-64 interrupt handling.
- Developing physical and virtual memory management.
- Implementing a keyboard driver, graphics driver, or interactive shell.
- Learning kernel debugging with QEMU and GDB.
- Integrating an existing DOOM port and understanding its platform requirements.

Responsibilities will be assigned according to individual interests, prior experience, and the project's technical requirements.

4. Risks and Countermeasures

Risk	Potential impact	Countermeasure
Technical complexity and debugging	Paging, interrupts, and boot-protocol issues may cause difficult-to-diagnose crashes.	Develop incrementally, maintain serial logging, use GDB, and test each subsystem independently.
Integration problems and unclear responsibilities	Incompatible interfaces and conflicting changes may delay the integrated kernel.	Define interfaces early, assign module ownership, review changes, and integrate continuously.
Time constraints and scope growth	Optional features and DOOM integration may consume time	Prioritize mandatory acceptance criteria, set a feature-freeze date, reserve

	needed for the core objectives.	debugging time, and treat DOOM as a stretch goal.

5. Immediate Next Steps

The team will begin by completing the following tasks:

1. Review the existing bootloader, kernel, common libraries, and VGA driver.
2. Identify which components are implemented and which have been tested successfully.
3. Confirm the mandatory objectives and the status of DOOM as a stretch goal.
4. Agree on the toolchain, directory structure, module interfaces, and coding conventions.
5. Assign initial responsibilities and individual learning objectives.
6. Establish a reproducible QEMU build and debugging workflow.
7. Convert the milestone weeks into calendar dates based on the semester deadline.
8. Create repository issues for the first milestone and document its completion evidence.

6. Research References

- **OSDev Wiki:** https://wiki.osdev.org/ - Reference material on boot protocols, x86-64 architecture, memory management, interrupts, and device drivers.
- **Limine Bootloader:** https://github.com/limine-bootloader/limine - Alternative bootloader documentation and reference material.
- **GNU GDB Documentation:** https://www.sourceware.org/gdb/documentation/ - Debugging and program inspection.
- **QEMU Documentation:** https://www.qemu.org/docs/master/ - Virtual machine configuration, serial devices, and debugging support.

The project will use the documentation corresponding to the selected toolchain and Multiboot2 boot configuration.

7. Expected Final Result

At the end of the semester, the team aims to deliver a reproducible, testable x86-64 kernel that boots in QEMU and provides an interactive shell, memory management, exception handling, timing, keyboard input, framebuffer graphics, and basic file access.

If the core objectives are completed with sufficient time remaining, the team will integrate an existing DOOM port to demonstrate the kernel's input, graphics, timing, memory, and file-loading capabilities.